Sounio: A Systems Programming Language for Epistemic Computing

Demetrios Chiuratto Agourakis*
Faculdade de Ciências Médicas e da Saúde, Pontifícia Universidade Católica de São Paulo (

Marli Gerenutti†
Faculdade de Ciências Médicas e da Saúde, Pontifícia Universidade Católica de São Paulo (

February 2026

Abstract

Scientific computing requires rigorous handling of measurement uncertainty, yet systems programming languages lack native support for propagating uncertainties in compliance with the Guide to the Expression of Uncertainty in Measurement (GUM, JCGM 100:2008). This leads to error-prone manual implementations and difficulties maintaining audit trails for regulatory contexts such as FDA 21 CFR Part 11 and ISO 17025.

We introduce **Sounio**, a systems programming language for *epistemic computing*—computation where uncertainty, confidence, and provenance are first-class type-level concerns. The `Knowledge<T>` type encapsulates a value, its GUM-compliant variance, a confidence measure, and a provenance chain for traceability. An algebraic effect system tracks epistemic operations, and the compiler warns when uncertainty is silently discarded.

Our contributions are: (1) a type system for epistemic values with formal GUM-compliance guarantees (Theorem 2); (2) a Scientific IR that represents uncertainty propagation as first-class compiler nodes; (3) provenance tracking with Merkle-hashed audit chains; and (4) cross-validation of compiled numerical code against analytical pharmacokinetic solutions, demonstrating first-order convergence. We illustrate the language design through applications in pharmacokinetics, climate modeling, and neuroimaging. The core compiler pipeline comprises approximately 319,000 lines of Rust (700,000 total including tooling and exploratory modules), with a 43,000-line self-hosted bootstrap in Sounio and a 236,000-line standard library spanning pharmacometrics, numerical methods, and autodifferentiation.

*ORCID: 0009-0008-2276-5847. Corresponding author: `demetrios@agourakis.med.br`
†ORCID: 0000-0001-7165-646X

Sounio is open source under the Apache License 2.0: `https://github.com/sounio-lang/sounio`

1 Introduction

Scientific computing frequently involves uncertain quantities: measured values with instrument precision, model parameters with confidence intervals, and predictions with error bounds. Yet mainstream programming languages treat numerical values as exact, forcing developers to either:

1. Ignore uncertainty (compromising scientific validity)
2. Track uncertainty manually (error-prone, tedious)
3. Use library wrappers (performance overhead, API friction)

The Guide to the Expression of Uncertainty in Measurement (GUM) [Joint Committee for Guides in Metrology, 2008] provides a rigorous framework for uncertainty propagation, but implementing GUM in general-purpose languages requires discipline that compilers cannot enforce.

We propose *epistemic computing* as a paradigm in which uncertainty, confidence, and provenance are native type-level constructs rather than library concerns. We introduce **Sounio**, a systems programming language that realizes this paradigm. The `Knowledge<T>` type carries not just a value but also:

- **Variance**: Uncertainty in the GUM sense (u^2)
- **Confidence**: A Beta distribution over credibility (library-level)
- **Provenance**: Audit trail for traceability

Arithmetic on `Knowledge<T>` automatically propagates uncertainty using first-order Taylor series (GUM linear method) or Monte Carlo sampling (GUM Supplement 1), with the choice configurable per-computation.

Contributions

- A type system for epistemic values with compile-time effect tracking and formal GUM-compliance proof (Section 3)
- GUM-compliant uncertainty propagation integrated into the compiler's intermediate representation (Section 4)
- Cross-validation of compiled numerical code against analytical solutions (Section 6)
- Illustrative case studies in pharmacokinetics, climate science, and neuroimaging (Section 7)

2 Background

2.1 The GUM Framework

The GUM [Joint Committee for Guides in Metrology, 2008] defines standard uncertainty $u(x)$ as the positive square root of variance:

$$u(x) = \sqrt{\mathrm{Var}(x)}$$

For a function $y = f(x_1, \ldots, x_N)$ of uncorrelated inputs, the *combined standard uncertainty* is:

$$u_c^2(y) = \sum_{i=1}^{N} \left(\frac{\partial f}{\partial x_i} \right)^2 u^2(x_i) \tag{1}$$

This first-order Taylor expansion (the "law of propagation of uncertainty") is exact for linear functions and approximate otherwise.

2.2 Existing Approaches

Python: uncertainties The `uncertainties` package [Lebigot, 2024] provides `ufloat` objects with automatic differentiation for Equation 1. However, operator overloading in Python incurs significant runtime overhead (see Section 6).

Julia: Measurements.jl Julia's `Measurements.jl` [Giordano, 2016] uses dual numbers for uncertainty tracking, achieving better performance through JIT compilation. However, it remains a library with no compile-time guarantees.

Domain-Specific Languages Languages like Stan [Carpenter et al., 2017] and BUGS model uncertainty probabilistically but are not general-purpose systems languages.

2.3 Limitations of Library Approaches

Library-based uncertainty tracking has three fundamental limitations:

1. **No static guarantees**: A function accepting `float64` can silently drop uncertainty.
2. **Runtime overhead**: Wrapper types require heap allocation, indirection, and runtime dispatch.
3. **No provenance**: Audit trails for regulatory compliance must be implemented separately.

Sounio addresses all three by making uncertainty a compile-time type property.

3 The Sounio Type System

3.1 The Knowledge Type

The core epistemic type is:

```
struct Knowledge<T> {
    value: T,                    // Point estimate
    variance: f64,               // Uncertainty (sigma^2)
    confidence: BetaConfidence,  // Credibility distribution
    provenance: Provenance,      // Audit trail
}
```

Listing 1: Knowledge type definition

`BetaConfidence` represents epistemic confidence as a Beta distribution, supporting Bayesian updating. In the current implementation, confidence handling is provided at the library level rather than enforced by the compiler's type checker:

```
struct BetaConfidence {
    alpha: f64,  // Successes + prior
    beta: f64,   // Failures + prior
}

fn beta_mean(c: &BetaConfidence) -> f64 {
    c.alpha / (c.alpha + c.beta)
}

fn beta_update(c: &BetaConfidence, successes: i64, failures: i64) -> BetaConfidence {
    BetaConfidence { alpha: c.alpha + successes, beta: c.beta + failures }
}
```

Listing 2: Confidence representation

3.2 Compiler Warnings for Uncertainty Loss

Sounio's type system prevents silent uncertainty loss through compile-time warnings. When a `Knowledge<T>` value is extracted to `T`, the compiler issues a diagnostic:

```
fn measure(instrument: &Instrument) -> Knowledge<f64> with IO {
    let raw = instrument.read()  // IO effect
    Knowledge::new(
        value: raw,
        variance: instrument.precision_squared(),
```

```
        confidence: BetaConfidence::from_calibration(
            instrument),
        provenance: Provenance::measurement(instrument)
    )
}

fn compute_exact(x: f64) -> f64 { x * 2.0 }

fn main() with IO {
    let k = Knowledge::measured(5.0, 0.01, "scale")
    let bad = compute_exact(k.value())  // Warning:
        uncertainty discarded
    let good = k.scale(2.0)              // OK:
        uncertainty preserved
}
```

Listing 3: Uncertainty extraction warning

3.3 Subtyping and Coercion

`Knowledge<T>` is not a subtype of `T`. Explicit extraction is required, generating compiler warnings when uncertainty is discarded:

Definition 1 (Epistemic Extraction). *The operation extract* : $\texttt{Knowledge<T>} \to \texttt{T}$ *is always permitted but generates a compiler warning at the call site to signal that uncertainty information is being discarded.*

3.4 Provenance Tracking

Every `Knowledge` value carries a `Provenance` chain:

```
enum Source {
    Measurement { instrument: String, timestamp: i64 },
    Computed { operation: String },
    Assertion { author: String },
    External { url: String, doi: Option<String> },
}

struct Provenance {
    sources: Vec<Source>,
    merkle_hash: [u8; 32],  // Structural hash (
        placeholder crypto)
}
```

Listing 4: Provenance structure

This provides structural support for audit trails relevant to regulated computational systems (e.g., FDA 21 CFR Part 11, ISO 17025), though full regulatory validation remains future work.

3.5 Formal Type System

We formalize the epistemic type system to ensure uncertainty is tracked and preserved. The typing judgment $\Gamma \vdash e : \texttt{Knowledge}\langle T \rangle$ asserts that expression e computes an epistemic value.

Definition 2 (Epistemic Typing). *The introduction rule for `Knowledge<T>`:*

$$\frac{\Gamma \vdash v : T \quad \Gamma \vdash u^2 : \texttt{f64} \quad \Gamma \vdash c : \texttt{BetaConf} \quad \Gamma \vdash p : \texttt{Prov}}{\Gamma \vdash \texttt{Knowledge::new}(v, u^2, c, p) : \texttt{Knowledge}\langle T \rangle}$$

3.6 Operational Semantics

We define small-step operational semantics for epistemic values before stating the type safety result that depends on them. A value is either a numeric constant n or a knowledge tuple $(v, \sigma^2, \beta, \pi)$ where v is the point estimate, σ^2 is variance, β is BetaConfidence, and π is provenance.

Definition 3 (Epistemic Evaluation). *The evaluation rules for arithmetic on knowledge values:*

$$\textit{E-ADD:} \quad (v_1, \sigma_1^2, \beta_1, \pi_1) + (v_2, \sigma_2^2, \beta_2, \pi_2) \\ \to (v_1{+}v_2, \sigma_1^2{+}\sigma_2^2, \mathit{comb}(\beta_1, \beta_2), \mathit{merge}(\pi_1, \pi_2)) \tag{2}$$

$$\textit{E-MUL:} \quad (v_1, \sigma_1^2, \beta_1, \pi_1) \times (v_2, \sigma_2^2, \beta_2, \pi_2) \\ \to (v_1 v_2, v_2^2\sigma_1^2{+}v_1^2\sigma_2^2, \mathit{comb}(\beta_1, \beta_2), \mathit{merge}(\pi_1, \pi_2)) \tag{3}$$

$$\textit{E-DIV:} \quad (v_1, \sigma_1^2, \beta_1, \pi_1)/(v_2, \sigma_2^2, \beta_2, \pi_2) \\ \to (v_1/v_2, \sigma_1^2/v_2^2{+}v_1^2\sigma_2^2/v_2^4, \ldots) \tag{4}$$

Theorem 1 (Type Safety). *If $\emptyset \vdash e : T$, then either e diverges or reduces to a value $v : T$ under the evaluation rules of Definition 2. If $T = \texttt{Knowledge}\langle S \rangle$, then v preserves all epistemic components without silent erasure.*

Proof sketch. By induction on the reduction relation (Definition 2). Epistemic operations are typed to require the `Epistemic` effect, and extraction requires explicit acknowledgment. Subtyping rules prevent implicit coercion from $\texttt{Knowledge}\langle T \rangle$ to T. Full proof in the companion technical report. □

The confidence combination function pools evidence under conditional independence:

$$\mathrm{comb}(\mathrm{Beta}(\alpha_1, \beta_1), \mathrm{Beta}(\alpha_2, \beta_2)) = \mathrm{Beta}(\alpha_1{+}\alpha_2{-}1, \beta_1{+}\beta_2{-}1)$$

4 Uncertainty Propagation

4.1 First-Order (Linear) Propagation

For arithmetic operations, Sounio implements Equation 1 at the IR level. Consider multiplication:

$$z = x \cdot y \tag{5}$$

$$u^2(z) = y^2 u^2(x) + x^2 u^2(y) \quad \text{(assuming independence)} \tag{6}$$

The compiler transforms:

```
let z = x * y  // where x, y: Knowledge<f64>
```

into:

```
let z_val = x.value * y.value
let z_var = y.value^2 * x.variance + x.value^2 * y.
    variance
let z = Knowledge { value: z_val, variance: z_var, ... }
```

4.2 Propagation Correctness

Theorem 2 (GUM Compliance)**.** *For any expression e composed of arithmetic operations on* `Knowledge` *values, let $\text{eval}(e) = (v, \sigma^2, \beta, \pi)$ under Sounio's operational semantics. Then $\sigma^2 = u_c^2(e)$ as defined by Equation 1, assuming uncorrelated inputs.*

Proof. By structural induction on e.

Base case: $e = \texttt{Knowledge::new}(v, \sigma^2, \beta, \pi)$. By definition, $\text{eval}(e) = (v, \sigma^2, \beta, \pi)$ and $u_c^2(e) = \sigma^2$. Thus $\sigma^2 = u_c^2(e)$.

Inductive case (addition): Let $e = e_1 + e_2$ where by IH, $\text{eval}(e_i) = (v_i, \sigma_i^2, \ldots)$ with $\sigma_i^2 = u_c^2(e_i)$. By E-ADD, $\text{eval}(e) = (v_1 + v_2, \sigma_1^2 + \sigma_2^2, \ldots)$. For $f(x, y) = x + y$, GUM gives $u_c^2 = (\partial f/\partial x)^2 \sigma_1^2 + (\partial f/\partial y)^2 \sigma_2^2 = \sigma_1^2 + \sigma_2^2$. ✓

Inductive case (multiplication): Let $e = e_1 \times e_2$. By E-MUL, $\text{eval}(e) = (v_1 v_2, v_2^2 \sigma_1^2 + v_1^2 \sigma_2^2, \ldots)$. For $f(x, y) = xy$, $\partial f/\partial x = y$ and $\partial f/\partial y = x$, so $u_c^2 = y^2 \sigma_1^2 + x^2 \sigma_2^2 = v_2^2 \sigma_1^2 + v_1^2 \sigma_2^2$. ✓

Division and transcendentals follow similarly via the delta method. □

4.3 Transcendental Functions

For $y = f(x)$, the delta method gives:

$$u^2(y) \approx \left(\frac{df}{dx} \right)^2 u^2(x)$$

Sounio provides built-in derivatives for standard functions:

Function	$f(x)$	$u^2(f(x))$
`sqrt`	$\sqrt{x}$	$\frac{u^2(x)}{4x}$
`exp`	e^x	$e^{2x}u^2(x)$
`ln`	$\ln x$	$\frac{u^2(x)}{x^2}$
`sin`	$\sin x$	$\cos^2(x)u^2(x)$
`cos`	$\cos x$	$\sin^2(x)u^2(x)$

4.4 Confidence Combination

When combining `Knowledge` values, confidence is updated using:

Proposition 1 (Confidence Product Rule)**.** *For independent measurements x and y with Beta-distributed confidences, the confidence of $z = f(x, y)$ is:*

$$\mathit{BetaConfidence}(z) = \mathit{combine}(\mathit{conf}(x), \mathit{conf}(y))$$

where combine pools evidence under the assumption of conditional independence.

4.5 Scientific IR (SIR) Design

Sounio's compiler includes a Scientific IR that represents epistemic operations as first-class nodes. The SIR is designed to enable domain-specific optimizations:

1. **Variance fusion**: Fusing $u^2(x + y + z)$ into a single pass
2. **Exact elision**: Skipping variance computation for `Knowledge::exact` (zero variance) operands
3. **Provenance elision**: Skipping Merkle hash computation when provenance is not required

These optimizations are designed into the SIR node structure; their implementation as automated compiler passes is ongoing work. The native backend includes SIMD support (AVX2 and NEON) for quaternion algebra, with extension to general epistemic arithmetic planned.

5 Implementation

5.1 Implementation Status

The v1.0.0-beta release provides native code generation for epistemic types, including GUM-compliant uncertainty propagation. A self-hosted compiler (43,000 lines of Sounio) implements a pipeline from lexer through native

x86-64 ELF generation via a three-stage bootstrap: a C virtual machine (Poseidon, 3,200 LOC) bootstraps the first self-hosted stage, which then compiles itself; IR normalization verifies stage equivalence. The standard library comprises 236,000 lines of Sounio covering scientific computing modules including autodiff, pharmacometric modeling, and numerical methods.

5.2 Compiler Architecture

Sounio's compiler repository contains approximately 700,000 lines of Rust, of which roughly 319,000 comprise the core compilation pipeline (frontend, type system, effect checker, IR passes, and four code generation backends), 71,000 cover developer tooling (LSP server, formatter, linter, REPL), and the remainder includes a scientific ontology, a PBPK modeling framework (DARWIN, 6,000 lines of Sounio), runtime libraries, and exploratory modules. The core pipeline is organized as:

Source → Lexer → Parser → AST → Type Check → HIR → SIR → HLIR → Backend

The key innovation is the **Scientific IR (SIR)**, an intermediate representation that treats epistemic operations as first-class nodes:

```
enum SirInst {
    PropagateAdd { lhs: NodeId, rhs: NodeId, result: NodeId },
    PropagateMul { lhs: NodeId, rhs: NodeId, result: NodeId },
    PropagateDiv { lhs: NodeId, rhs: NodeId, result: NodeId },
}
```

Listing 5: SIR epistemic instructions (design)

5.3 Planned SIR Optimization Passes

The SIR node structure is designed to enable the following optimization passes, which are specified but not yet implemented as automated compiler transforms:

Variance Fusion Consecutive epistemic additions can be fused into a single variance computation:

$$\texttt{t1 = x + y; t2 = t1 + z} \Rightarrow \texttt{t2 = x + y + z}$$
$$(2 \text{ variance computations} \rightarrow 1 \text{ fused: } \sigma_x^2 + \sigma_y^2 + \sigma_z^2)$$

Exact Elision Operations involving `Knowledge::exact` (zero variance) can skip variance propagation entirely, reducing to scalar arithmetic.

Provenance Elision When provenance tracking is not required, Merkle hash computation can be skipped, reducing overhead for non-audited computations.

5.4 Memory Layout

`Knowledge<f64>` occupies 40 bytes, stack-allocated:

Field	Size
`value`	8 bytes (f64)
`variance`	8 bytes (f64)
`confidence`	16 bytes (BetaConf)
`provenance`	8 bytes (pointer)

The 40-byte layout is designed for SIMD-friendly access; the native backend currently uses SIMD (AVX2, AVX-512, NEON) for quaternion and octonion algebra, with extension to general epistemic arithmetic planned.

6 Evaluation

6.1 Performance Considerations

Sounio's design offers several structural advantages over library-based uncertainty propagation:

1. **Stack allocation**: `Knowledge<f64>` is a 40-byte stack-allocated value, avoiding the heap allocation and indirection required by Python's `ufloat` wrapper objects.
2. **Static dispatch**: All epistemic operations are monomorphized at compile time, eliminating the runtime dispatch overhead of operator overloading in dynamic languages.
3. **Compiler-visible structure**: Because uncertainty propagation is part of the IR, the compiler can in principle apply domain-specific optimizations (Section 4) that are unavailable to library approaches.

Cross-language benchmark scripts (Python `uncertainties`, Julia `Measurements.jl`, and Sounio) are included in the repository under `benches/misc/uncertainty/`. Table 1 reports median wall-clock times over 5–10 runs after warmup on a Xeon Gold 6148 @ 2.40 GHz. Sounio timings are from the tree-walking interpreter; JIT and native backends would reduce these further.

Table 1: Uncertainty propagation benchmarks (median ms). "DNF" = did not finish within 600 s. †Derivative accumulation causes $O(n^2)$ runtime on long chains. ‡Extrapolated from 1K benchmark.

Benchmark	Python `uncertainties`	Julia `Measurements.jl`	Sounio `Knowledge<T>`
GUM Chain (10K ops)	137	DNF†	**29**
GUM Chain (100K ops)	1 804	DNF†	**237**
Monte Carlo (10K)	652	**5**	51
Monte Carlo (100K)	6 496	**49**	455
Cockcroft–Gault (10K)	606	DNF†	**71**
Transcendental (10K)	584	**4**‡	53

Scaling analysis The primary finding concerns asymptotic behavior, not absolute speed. Julia's `Measurements.jl` tracks exact derivatives with respect to every independent measurement source. For independent-per-iteration workloads (Monte Carlo), derivative dictionaries remain small and JIT-compiled Julia achieves 5 ms per 10K iterations. However, when a single value accumulates operations across many steps—the typical pattern in ODE solvers, control loops, and cumulative clinical computations—the derivative dictionary grows at each step, causing $O(n^2)$ total runtime (Cockcroft–Gault at 1K patients: 7.6 s; at 10K: >600 s). Python's `uncertainties` uses a similar derivative-tracking strategy but with CPython overhead. Sounio's `Knowledge<T>` propagates epistemic metadata in $O(1)$ per operation regardless of chain length, achieving linear scaling on all workloads.

6.2 Numerical Accuracy: ODE Solver Cross-Validation

To validate Sounio's numerical computing infrastructure beyond microbenchmarks, we cross-validate against the analytical Bateman equation for a two-compartment pharmacokinetic model. This tests both floating-point arithmetic accuracy and long-running loop execution with struct mutation (up to 2,400 iterations), exercising the compiler's SSA construction, phi-node insertion, and backend codegen on a realistic scientific workload.

Model The two-compartment oral absorption model has parameters: Dose = 500 mg, absorption rate $k_a = 1.0\,\mathrm{h}^{-1}$, elimination rate $k_e = 0.1\,\mathrm{h}^{-1}$. The closed-form Bateman solution [Gibaldi and Perrier, 1982] gives:

$$C_{\text{gut}}(t) = D \cdot e^{-k_a t} \tag{7}$$

$$C_{\text{central}}(t) = \frac{D \cdot k_a}{k_a - k_e}\left(e^{-k_e t} - e^{-k_a t}\right) \tag{8}$$

Results The test suite validates Sounio's forward Euler solver at two step sizes ($\Delta t = 0.1\,\text{h}$ and $\Delta t = 0.01\,\text{h}$) against the analytical Bateman solution. Integration tests assert that all intermediate checkpoints at $t \in \{2, 6, 12, 24\}\,\text{h}$ fall within 5% of the analytical values, consistent with the $\mathcal{O}(\Delta t)$ global error of forward Euler.

The analytical solution at $t = 24\,\text{h}$ gives $C_{\text{central}} \approx 50.4\,\text{mg}$. Both solver configurations pass the 5% tolerance tests, and reducing Δt by $10\times$ correspondingly reduces the observed error, confirming first-order convergence and validating the compiler's numerical fidelity across multi-thousand-iteration loops.

Pharmacokinetic Parameters The Euler solver ($\Delta t = 0.1\,\text{h}$, 240 steps) recovers the expected PK profile within test tolerances: $C_{\max}$ in the range $[100, 400]\,\text{mg}$ (analytical: $C_{\max} \approx 387\,\text{mg}$), $\text{AUC}_{0\to 24}$ in $[3000, 6000]\,\text{mg}\cdot\text{h}$ (theoretical $\text{AUC}_{0\to\infty} = D/k_e = 5000\,\text{mg}\cdot\text{h}$). All integration tests pass in the CI suite.

Higher-Order Integration A three-compartment model (Gut $\to$ Liver $\to$ Central) with hepatic first-pass metabolism ($f_a = 0.8$, $Q_h = 0.5$, $\text{CL}_h = 0.3$) was validated using the RK2 midpoint method (480 steps, $\Delta t = 0.05\,\text{h}$). Conservation-of-mass invariants hold: concentrations remain non-negative, and pharmacokinetic endpoints ($C_{\max}$, $T_{\max}$, AUC) fall within physiologically plausible ranges verified by automated tests.

These results demonstrate that Sounio's native backend produces correct numerical code for sustained ODE simulations, a prerequisite for the epistemic ODE integration described in the companion formalization (Section 9).

6.3 Compile-Time Overhead

Epistemic type checking adds overhead to compilation from variance propagation analysis and provenance chain construction. For non-epistemic code paths, there is no additional compile-time overhead since the epistemic passes are not invoked. Quantitative measurement of compile-time overhead is deferred to the benchmarking follow-up.

7 Applications

The following examples illustrate Sounio's language design in three scientific domains. These are representative programs showing how epistemic types express domain concerns; the pharmacokinetics example is validated by the ODE cross-validation tests described above, while the climate and neuroimaging examples demonstrate language expressiveness.

7.1 Pharmacokinetics

The Cockcroft-Gault equation for creatinine clearance:

$$\text{CrCl} = \frac{(140 - \text{age}) \times \text{weight} \times [0.85 \text{ if female}]}{72 \times \text{SCr}}$$

In clinical practice, each input has measurement uncertainty. Sounio automatically propagates these to the output:

```
fn creatinine_clearance(
    age: Knowledge<years>,
    weight: Knowledge<kg>,
    scr: Knowledge<mg_per_dL>,
    is_female: bool
) -> Knowledge<mL_per_min> {
    let factor = if is_female { 0.85 } else { 1.0 }
    let numerator = (Knowledge::exact(140.0) - age) *
        weight
    numerator.scale(factor) / (Knowledge::exact(72.0) *
        scr)
}
```

Listing 6: GUM-compliant drug dosing

The result includes propagated uncertainty with provenance metadata, which can support documentation workflows in regulated environments.

Beyond individual PK calculations, Sounio's standard library includes DARWIN, a 6,000-line PBPK modeling framework implementing 14-compartment mammalian physiology with Rodgers–Rowland tissue partitioning [Rodgers et al., 2005], allometric scaling, and mechanistic drug–drug interaction modeling via CYP inhibition. A companion MedLang DSL (8,100 lines with lexer, parser, AST, and code generator) provides a domain-specific notation for population pharmacometric models.

7.2 Climate Modeling

CMIP6 climate projections involve multiple sources of uncertainty: model structural uncertainty, parameter uncertainty, and natural variability. Sounio's epistemic types enable rigorous ensemble analysis:

```
fn sea_level_rise_projection(
    models: &[Knowledge<meters>],  // Each model has
        internal uncertainty
    scenario: EmissionScenario
) -> Knowledge<meters> {
    // Within-model uncertainty from each projection
    var within_model = 0.0
    for m in models { within_model = within_model + m.
        variance }
```

```
    within_model = within_model / models.len()

    // Between-model uncertainty from ensemble spread
    let mean_val = Knowledge::mean_value(models)
    var between_model = 0.0
    for m in models {
        let diff = m.value - mean_val
        between_model = between_model + diff * diff
    }
    between_model = between_model / (models.len() - 1)

    Knowledge::new(
        value: mean_val,
        variance: within_model + between_model,  // Total uncertainty
        confidence: BetaConfidence::from_ensemble_size(models.len()),
        provenance: Provenance::cmip6(scenario)
    )
}
```

Listing 7: Climate ensemble analysis with structured uncertainty

This approach structures uncertainty in a way compatible with IPCC conventions for reporting ranges in climate projections, where "likely" (66%) and "very likely" (90%) ranges can be derived from the confidence distribution.

7.3 Neuroimaging

fMRI statistical analysis requires careful propagation of scanner noise through preprocessing pipelines. Sounio tracks uncertainty through motion correction, smoothing, and GLM estimation:

```
fn preprocess_fmri(
    raw: &[Knowledge<VoxelIntensity>],
    motion_params: &MotionEstimate
) -> Vec<Knowledge<VoxelIntensity>> with IO {
    let corrected = motion_correct(raw, motion_params)
    let smoothed = gaussian_smooth(corrected, fwhm: 6.0)
    // Uncertainty accumulates through pipeline
    // Final variance includes: scanner noise + motion residual + smoothing
    smoothed
}

fn first_level_glm(
    timeseries: &[Knowledge<f64>],
    design: &DesignMatrix
) -> Knowledge<BetaCoefficient> {
```

```
    // GLM with uncertainty-weighted least squares
    Knowledge::weighted_regression(timeseries, design)
}
```

Listing 8: fMRI preprocessing with uncertainty

This illustrates how scanner measurement uncertainty could be tracked through to final statistical maps, with provenance metadata supporting reproducibility and data sharing.

8 Related Work

Probabilistic Programming Languages Stan [Carpenter et al., 2017] employs Hamiltonian Monte Carlo for Bayesian inference in statistical modeling. Pyro [Bingham et al., 2019] extends deep probabilistic programming to PyTorch, while Gen [Cusumano-Towner et al., 2019] introduces programmable inference. Turing.jl [Ge et al., 2018] provides universal probabilistic programming in Julia with composable inference. These are domain-specific languages for Bayesian inference rather than systems programming languages with native uncertainty types. Sounio's `Knowledge<T>` tracks frequentist (GUM) uncertainty at the type level, complementing rather than replacing Bayesian approaches; bridges from MCMC posteriors and conformal prediction intervals to `Knowledge` values are provided in the standard library.

Uncertainty Propagation Libraries Python's `uncertainties` [Lebigot, 2024] uses operator overloading but incurs runtime overhead. Julia's `Measurements.jl` [Giordano, 2016] leverages dual numbers with JIT compilation for better performance. Both are libraries with no compile-time guarantees on uncertainty preservation.

Dependent and Refinement Types Idris [Brady, 2013] supports full dependent types for proving properties like totality. F* [Swamy et al., 2016] uses refinement types for verification. These systems model correctness properties but not epistemic uncertainty semantically.

Units of Measure F#'s units of measure [Kennedy, 2009] provide compile-time dimensional analysis. Sounio extends this paradigm by combining units with epistemic uncertainty, allowing types to represent both physical dimensions and associated measurement errors.

Automatic Differentiation JAX [Bradbury et al., 2018] and PyTorch [Paszke et al., 2019] compute gradients via AD. Sounio adapts similar trans-

formation techniques for uncertainty propagation, treating uncertainty as a first-class type-level concern rather than a numerical derivative.

Interval Arithmetic Libraries like MPFI and Arb compute conservative bounds via interval arithmetic. Unlike intervals that bound worst-case errors, GUM models statistical uncertainty with probabilistic distributions, providing tighter estimates for real-world measurements.

9 Formal Metatheory

We formalize the epistemic type system with full operational semantics and prove three key properties in a companion technical report:

1. **Progress:** If $\emptyset \vdash e : \tau$, then e is a value or $e \longrightarrow e'$.
2. **Preservation:** If $\Gamma \vdash e : \tau$ and $e \longrightarrow e'$, then $\Gamma \vdash e' : \tau$.
3. **GUM Soundness:** For any well-typed expression $e : \texttt{Knowledge}\langle T, \epsilon \rangle$ that reduces to value (v, u), the uncertainty u equals the GUM combined standard uncertainty u_c (Eq. 1) for uncorrelated inputs.

The formalization extends to ODE integration, where an epistemic degradation rule tracks numerical uncertainty growth:

$$\epsilon(t + \Delta t) = \epsilon(t) \cdot (1 + \alpha \cdot \Delta t)$$

with α depending on the solver method. The cross-validation results in Section 6 validate this model empirically.

10 Conclusion

Sounio demonstrates that uncertainty quantification can be integrated into a systems programming language with strong static guarantees. The `Knowledge<T>` type makes GUM-compliant uncertainty propagation as natural as arithmetic, while the algebraic effect system warns when uncertainty is silently discarded. Cross-validation against analytical solutions confirms the numerical fidelity of the compiled code across multi-thousand-iteration ODE simulations.

The standard library provides extensions beyond the core type system described here, including an EpistemicDual type for forward-mode AD with uncertainty propagation (590 lines), the DARWIN PBPK framework and MedLang DSL described in Section 7, and compile-time causal identifiability checking via Z3 SMT solving (integrated into the dependent type checker). These are implemented but not described in detail in this paper; we plan separate publications for each.

Future work includes:

- Implementing SIR optimization passes (variance fusion, exact/provenance elision)
- Mechanized metatheory proofs in Coq or Lean
- Full regulatory validation for FDA 21 CFR Part 11 compliance
- Distributed epistemic computing across heterogeneous nodes

Availability Sounio is open source under the Apache License 2.0.

- Repository: `https://github.com/sounio-lang/sounio`
- Documentation: `https://souniolang.org`
- DOI: 10.5281/zenodo.18404188

References

Eli Bingham, Jonathan P Chen, Martin Jankowiak, Fritz Obermeyer, Neeraj Pradhan, Theofanis Karaletsos, Rohit Singh, Paul Szerlip, Paul Horsfall, and Noah D Goodman. Pyro: Deep universal probabilistic programming. *Journal of Machine Learning Research*, 20(28):1–6, 2019. doi:10.48550/arXiv.1810.09538. URL `http://jmlr.org/papers/v20/18-403.html`.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL `https://github.com/jax-ml/jax`.

Edwin Brady. Idris, a general-purpose dependently typed programming language: Design and implementation. *Journal of Functional Programming*, 23(5):552–593, 2013. doi:10.1017/S095679681300018X.

Bob Carpenter, Andrew Gelman, Matthew D Hoffman, Daniel Lee, Ben Goodrich, Michael Betancourt, Marcus Brubaker, Jiqiang Guo, Peter Li, and Allen Riddell. Stan: A probabilistic programming language. *Journal of Statistical Software*, 76(1), 2017. doi:10.18637/jss.v076.i01.

Marco F Cusumano-Towner, Feras A Saad, Alexander K Lew, and Vikash K Mansinghka. Gen: A general-purpose probabilistic programming system with programmable inference. In *PLDI*, 2019. doi:10.1145/3314221.3314642.

Hong Ge, Kai Xu, and Zoubin Ghahramani. Turing: A language for flexible probabilistic inference. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 1682–1690, 2018.

Milo Gibaldi and Donald Perrier. *Pharmacokinetics*. Marcel Dekker, 2nd edition, 1982. ISBN 978-0824710422.

Mosè Giordano. Uncertainty propagation with functionally correlated quantities. 2016. doi:10.48550/arXiv.1610.08716. URL `https://github.com/JuliaPhysics/Measurements.jl`.

Joint Committee for Guides in Metrology. JCGM 100:2008 – Evaluation of measurement data – Guide to the expression of uncertainty in measurement (GUM). Technical report, BIPM, 2008. URL `https://www.bipm.org/en/doi/10.59161/jcgm100-2008e`.

Andrew Kennedy. Types for units-of-measure: Theory and practice. In *CEFP Summer School*, 2009. doi:10.1007/978-3-642-17685-2_8.

Eric O. Lebigot. Uncertainties: a Python package for calculations with uncertainties, 2024. URL `https://github.com/lebigot/uncertainties`.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *NeurIPS*, 2019. doi:10.48550/arXiv.1912.01703.

Trudy Rodgers, David Leahy, and Malcolm Rowland. Physiological solubility of drug-like molecules for ivive tissue distribution predictions. *Journal of Pharmaceutical Sciences*, 94(6):1259–1276, 2005. doi:10.1002/jps.20322.

Nikhil Swamy, Cătălin Hriţcu, Chantal Keller, Aseem Rastogi, Antoine Delignat-Lavaud, Simon Forest, Karthikeyan Bhargavan, Cédric Fournet, Pierre-Yves Strub, Markulf Kohlweiss, et al. Dependent types and multimonadic effects in F*. In *POPL*, 2016. doi:10.1145/2837614.2837655.